

Compte rendu de TP

Big Data – Module 03 : Cloud Storage & Object Storage

Lab 1 : projet Python avec uv • Lab 2 : stockage objet S3

Auteur	Dorline TIAKO OKASSIE
Promotion	ING4 – Groupe 01 – Campus de Paris
Environnement	Onyxia (Datalab Adaltas) – service vscode-pyspark
Stockage	SeaweedFS (API S3) – bucket user-d-tiakookassie-ece
Dépôt Git	github.com/Dorline0121/ece-2026-bigdata
Date	2 octobre 2026

Introduction

Ce compte rendu présente la réalisation des deux travaux pratiques du module **Cloud Storage and Object Storage**. Le **lab 1** met en place un projet Python géré avec **uv** qui génère deux jeux de données liés (utilisateurs et commandes). Le **lab 2** exploite ces données avec un stockage objet compatible **S3** : opérations de base, métadonnées, URL présignées, upload multipart, versioning, règles de cycle de vie, contrôle d'accès, puis ingestion dans la couche *bronze* depuis un **Job Kubernetes**.

Tous les travaux ont été réalisés dans VS Code lancé depuis le portail Onyxia (Datalab Adaltas). Pour chaque étape sont donnés l'objectif, les commandes principales, le résultat obtenu et, le cas échéant, l'écart avec l'énoncé.

Synthèse des résultats

Élément	Résultat attendu	Résultat obtenu
Projet uv + scripts	dataset-users / dataset-orders fonctionnels	Conforme : 1er utilisateur « Charles Garcia », 1re commande « cookie »
Dépôt GitHub	Historique de commits poussé	4 commits sur main (Dorline0121/ece-2026-bigdata)
Upload / download S3	Fichier identique après aller-retour	« identical », 7 351 octets
ETag vs MD5	ETag = MD5 (upload simple)	6a28331508e96326f4ae39eac53d02a8 identiques
Multipart 200 Mio	ETag suffixé -25	a9bf...98d7-25
Versioning	Versions multiples + delete marker	3 versions conservées + 1 delete marker
Bucket policy	Suppression refusée ?	Oui : AccessDenied
Job Kubernetes	bronze/users.csv et bronze/orders.csv	7 351 et 331 568 octets

Lab 1 - Projet Python avec uv et génération de jeux de données

Objectifs : créer et gérer un projet Python avec uv, puis générer des jeux de données aléatoires mais reproductibles d'utilisateurs et de commandes. Durée indicative : 1 h.

Étape 1 - Service vscode-pyspark et terminal

Un service **vscode-pyspark** a été lancé depuis le catalogue Onyxia. Dans la section *Role*, l'accès au namespace a été activé avec le rôle Kubernetes **admin**, nécessaire à la dernière partie du lab 2 (création de Jobs et de Secrets).

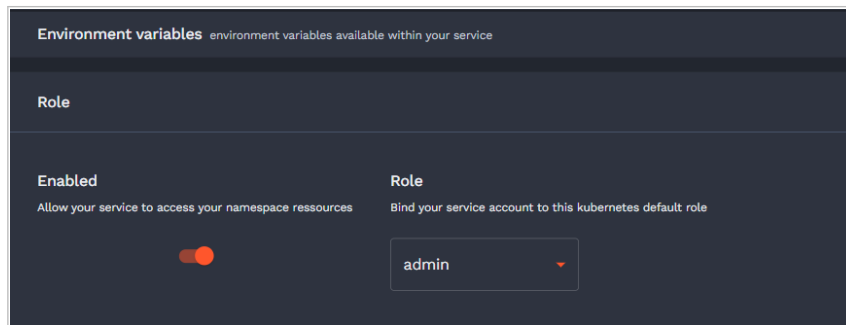


Figure 1 - Configuration du service : rôle Kubernetes « admin »

Vérification de l'environnement : répertoire de travail, présence de uv, de la CLI aws et de kubectl, et droits Kubernetes.

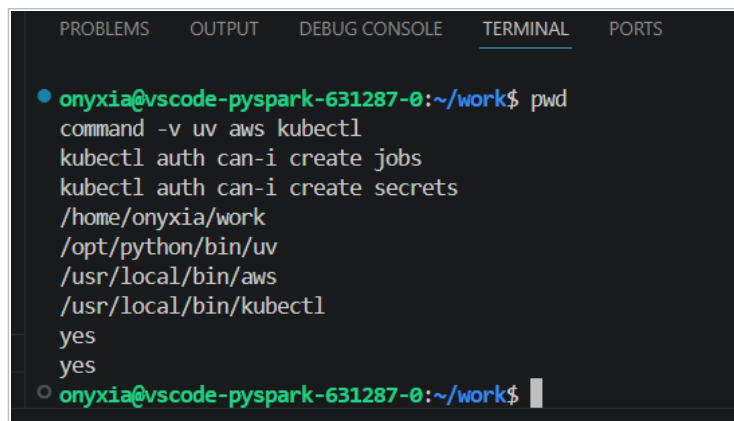


Figure 2 - Outils disponibles et droits « create jobs / create secrets » accordés

Étape 2 - Initialisation Git et projet uv

```
git config --global init.defaultBranch main
git init
cat <<'INI' >.gitignore
.*
!.gitignore
!.python-version
__pycache__/*
*.csv
INI
uv init --package
uv sync
```

uv init --package crée un projet installable : pyproject.toml, .python-version, README.md et le package src/work/. **uv sync** crée l'environnement virtuel .venv et le fichier de verrouillage uv.lock, versionné pour reproduire exactement les mêmes dépendances. La règle *.csv, demandée au lab 2, a été ajoutée dès le début : les jeux de données ne sont pas du code source.

```

uv init --package
uv sync
ls -a ...

[project.scripts]
work = "work:main"

[build-system]
requires = ["uv_build>=0.12.10,<0.13.0"]
build-backend = "uv_build"

```

Figure 3 – pyproject.toml généré : sections [project.scripts] et [build-system]

Étape 3 – Premier commit et publication sur GitHub

Difficulté : le premier commit a échoué avec detected dubious ownership. Le dossier /home/onyxia/work est monté par la plateforme et n'appartient pas à l'utilisateur courant ; Git refuse alors d'y travailler par sécurité.

```

onyxia@vscode-pyspark-631287-0:~/work$ git add .gitignore .python-version pyproject.toml README.md src uv.lock
git commit -m "feat: initial project layout"
fatal: detected dubious ownership in repository at '/home/onyxia/work'
To add an exception for this directory, call:

    git config --global --add safe.directory /home/onyxia/work
fatal: detected dubious ownership in repository at '/home/onyxia/work'
To add an exception for this directory, call:

    git config --global --add safe.directory /home/onyxia/work
onyxia@vscode-pyspark-631287-0:~/work$

```

Figure 4 – Erreur « dubious ownership » au premier commit

Correction : déclarer explicitement le dossier comme sûr, puis relancer le commit.

```

git config --global --add safe.directory /home/onyxia/work
git add .gitignore .python-version pyproject.toml README.md src uv.lock
git commit -m "feat: initial project layout"

```

```

onyxia@vscode-pyspark-631287-0:~/work$ git config --global --add safe.directory /home/onyxia/work
git status | head -1
git add .gitignore .python-version pyproject.toml README.md src uv.lock
git commit -m "feat: initial project layout"
create mode 100644 .gitignore
create mode 100644 .python-version
create mode 100644 README.md
create mode 100644 pyproject.toml
create mode 100644 src/work/__init__.py
create mode 100644 uv.lock

```

Figure 5 – Premier commit créé

Le premier push a renvoyé Repository not found : l'URL du dépôt distant utilisait un nom de compte erroné. Le nom réel du compte, **Dorline0121**, a été relevé sur la page du dépôt, puis l'URL corrigée avec git remote set-url.

```

onyxia@vscode-pyspark-631287-0:~/work$ git remote add origin "https://github.com/Dorline21/ece-2026-bigdata.git"
git push -u origin main

remote: Repository not found.
fatal: repository 'https://github.com/Dorline21/ece-2026-bigdata.git/' not found

```

Figure 6 – Push refusé : dépôt introuvable à l'URL initiale

```
git remote -v
git push -u origin main
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (10/10), 977 bytes | 325.00 KiB/s, done.
Total 10 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Dorline0121/ece-2026-bigdata.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Figure 7 – Push réussi après correction de l'URL ; branche main suivie

Étape 4 – README, sérialisation et dépendance Faker

Le README a été complété (description et usage), puis committé. Le module `serialize.py` imprime une liste de dictionnaires en **json**, **jsonline** (un document JSON par ligne) ou **csv**. La bibliothèque Faker a été ajoutée avec `uv add faker`, qui l'inscrit à la fois dans `pyproject.toml` et dans `uv.lock`.

```
onxyia@vscode-pyspark-631287-0:~/work$ cat <<'PY'>src/work/serialize.py
import csv
import io
import json ...
Installed 2 packages in 58ms
+ faker==40.40.0
~ work==0.1.0 (from file:///home/onxyia/work)
dependencies = [
  "faker>=40.40.0",
]
```

Figure 8 – Ajout de faker 40.40.0 aux dépendances

Étape 5 – Scripts de génération

dataset_users.py génère des utilisateurs (uuid + profil Faker). **dataset_orders.py** génère, pour chaque utilisateur, entre *count-min* et *count-max* commandes reliées par *user_uuid* (équivalent d'une clé étrangère), avec un produit tiré d'un *DynamicProvider*, une quantité et une date avançant d'une heure par commande à partir du 1^{er} janvier 2020. `Faker.seed(42)` rend la génération déterministe.

Pour contrôler les copier-coller dans le terminal, l'intégrité des fichiers a été vérifiée par empreinte MD5 par rapport à une version de référence testée au préalable.

```
onxyia@vscode-pyspark-631287-0:~/work$ wc -l src/work/serialize.py src/work/dataset_users.py
md5sum src/work/dataset_users.py
uv run python -m py_compile src/work/dataset_users.py && echo "syntaxe OK"
 22 src/work/serialize.py
 38 src/work/dataset_users.py
 60 total
1b06e02f7a2d153798e75e0702c94470  src/work/dataset_users.py
syntaxe OK

onxyia@vscode-pyspark-631287-0:~/work$ cat <<'PY'>src/work/dataset_orders.py
import argparse
import datetime
...
from work.serialize import serialize

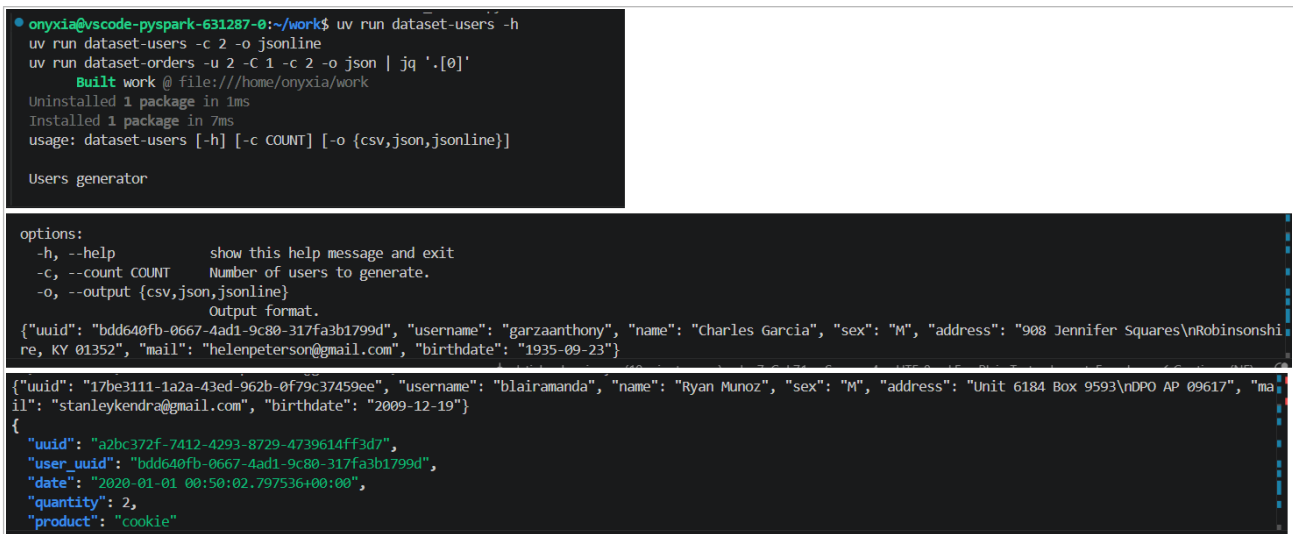
fake = Faker()
md5sum src/work/dataset_orders.pyn, args.count_max, args.count_users, args.date_from, args.output)
80 src/work/dataset_orders.py
caf30e27fa977f9d80c8b9e8cb3f6b54  src/work/dataset_orders.py
```

Figure 9 – Contrôle des fichiers : 38 lignes / 1b06e02f... et 80 lignes / caf30e27..., syntaxe valide

Étape 6 – Points d'entrée et exécution

Dans [project.scripts], l'entrée par défaut a été remplacée par les deux commandes du projet :

```
[project.scripts]
dataset-users = "work.dataset_users:main"
dataset-orders = "work.dataset_orders:main"
```



```
onyxia@vscode-pyspark-631287-0:~/work$ uv run dataset-users -h
uv run dataset-users -c 2 -o jsonline
uv run dataset-orders -u 2 -C 1 -c 2 -o json | jq '.[0]'
Built work @ file:///home/onyxia/work
Uninstalled 1 package in 1ms
Installed 1 package in 7ms
usage: dataset-users [-h] [-c COUNT] [-o {csv,json,jsonline}]

Users generator

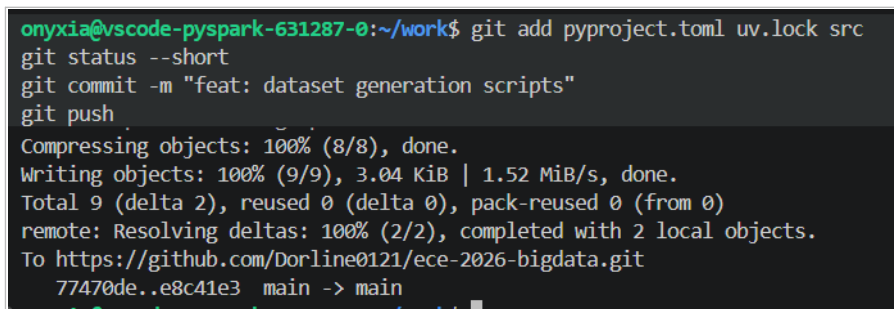
options:
  -h, --help            show this help message and exit
  -c, --count COUNT      Number of users to generate.
  -o, --output {csv,json,jsonline}
                        Output format.

{"uuid": "bdd640fb-0667-4ad1-9c80-317fa3b1799d", "username": "garzaanthony", "name": "Charles Garcia", "sex": "M", "address": "908 Jennifer Squares\nRobinsonshi", "re, KY 01352", "mail": "helenpeterson@gmail.com", "birthdate": "1935-09-23"}

{"uuid": "17be3111-1a2a-43ed-962b-0f79c37459ee", "username": "blairamanda", "name": "Ryan Munoz", "sex": "M", "address": "Unit 6184 Box 9593\nDPO AP 09617", "ma", "il": "stanleykendra@gmail.com", "birthdate": "2009-12-19"}
{
  "uuid": "a2bc372f-7412-4293-8729-4739614ff3d7",
  "user_uuid": "bdd640fb-0667-4ad1-9c80-317fa3b1799d",
  "date": "2020-01-01 00:50:02.797536+00:00",
  "quantity": 2,
  "product": "cookie"
```

Figure 10 – Exécution : aide de dataset-users, deux utilisateurs, et première commande liée à l'uuid bdd640fb... (Charles Garcia)

Résultat : les sorties sont identiques à l'énoncé. Le user_uuid de la première commande correspond à l'uuid du premier utilisateur, ce qui confirme le lien entre les deux jeux de données.



```
onyxia@vscode-pyspark-631287-0:~/work$ git add pyproject.toml uv.lock src
git status --short
git commit -m "feat: dataset generation scripts"
git push
Compressing objects: 100% (8/8), done.
Writing objects: 100% (9/9), 3.04 KiB | 1.52 MiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Dorline0121/ece-2026-bigdata.git
77470de..e8c41e3 main -> main
```

Figure 11 – Commit final « feat: dataset generation scripts » poussé

Lab 2 - Stockage objet avec S3

Objectifs : manipuler un bucket S3 (upload, liste, téléchargement, suppression), utiliser les métadonnées, les URL présignées, l'upload multipart, le versioning, les règles de cycle de vie et le contrôle d'accès, puis déposer les jeux de données dans la couche *bronze* depuis un Job Kubernetes. Le stockage de la plateforme est **SeaweedFS**, compatible avec l'API S3.

Étape 1 - Configuration de l'accès S3

Difficulté : toutes les commandes de l'énoncé utilisent `--profile 'default'`, mais ce profil n'existait pas : `~/aws` ne contenait qu'un cache, sans fichiers config ni credentials. Les identifiants temporaires étaient en revanche fournis par des variables d'environnement `AWS_*`.

```
onyxia@vscode-pyspark-631287-0:~/work$ aws configure list --profile 'default'
export S3_ENDPOINT_URL=$(
  aws configure get endpoint_url --profile 'default' \
  || echo "https://$AWS_S3_ENDPOINT" ...

aws: [ERROR]: The config profile (default) could not be found

aws: [ERROR]: The config profile (default) could not be found
https://s3.seaweedfs.adm.adaltas.cloud
user-d-tiakookassie-ece

onyxia@vscode-pyspark-631287-0:~/work$ ls -la ~/.aws 2>&1
total 16
drwxr-xr-x 3 onyxia users 4096 Oct  2 10:17 .
drwxr-xr-x 1 onyxia users 4096 Oct  2 10:17 ..
drwxr-xr-x 3 onyxia users 4096 Oct  2 10:17 cli

onyxia@vscode-pyspark-631287-0:~/work$ ls -la ~/.aws 2>&1
env | grep '^AWS_' | cut -d= -f1
AWS_S3_ENDPOINT
AWS_DEFAULT_REGION
AWS_WORKING_DIRECTORY_PATH
AWS_SECRET_ACCESS_KEY
AWS_ACCESS_KEY_ID
AWS_ENDPOINT_URL
AWS_SESSION_TOKEN
AWS_PATH_STYLE_ACCESS
```

Figure 12 - Profil « default » introuvable ; variables `AWS_*` présentes

Correction : création du profil à partir des variables, sans jamais afficher les secrets. Le style d'adressage *path* est imposé (URL `https://endpoint/bucket/clé`), comme attendu par SeaweedFS. L'endpoint a été défini sans « / » final pour éviter des URL invalides avec `s5cmd` et `boto3`.

```
aws configure set aws_access_key_id "$AWS_ACCESS_KEY_ID" --profile default
aws configure set aws_secret_access_key "$AWS_SECRET_ACCESS_KEY" --profile default
aws configure set aws_session_token "$AWS_SESSION_TOKEN" --profile default
aws configure set region "${AWS_DEFAULT_REGION:-us-east-1}" --profile default
aws configure set endpoint_url "https://$AWS_S3_ENDPOINT" --profile default
aws configure set s3.addressing_style path --profile default
export S3_ENDPOINT_URL=$(aws configure get endpoint_url --profile 'default')
export LAB_BUCKET_NAME="$KUBERNETES_NAMESPACE"
```

```
onyxia@vscode-pyspark-631287-0:~/work$ aws configure set endpoint_url "https://$AWS_S3_ENDPOINT" --profile default
export S3_ENDPOINT_URL=$(aws configure get endpoint_url --profile 'default')
echo "$S3_ENDPOINT_URL"
aws configure list --profile 'default' ...
https://s3.seaweedfs.adm.adaltas.cloud
NAME      : VALUE                                : TYPE      : LOCATION
profile   : default                                : manual    : --profile
access_key : *****1215                            : shared-credentials-file :
secret_key : *****ICV=                            : shared-credentials-file :
region    : us-east-1                              : env       : ['AWS_REGION', 'AWS_DEFAULT_REGION']
2026-09-10 15:32:21 open-data
```

Figure 13 - Profil opérationnel : clés masquées lues depuis le fichier d'identifiants

Bucket personnel : le bucket user-d-tiakookassie-ece n'existait pas encore. Il a été créé depuis la page **Stockage de données** du portail Onyxia, puis son accès vérifié depuis le terminal.

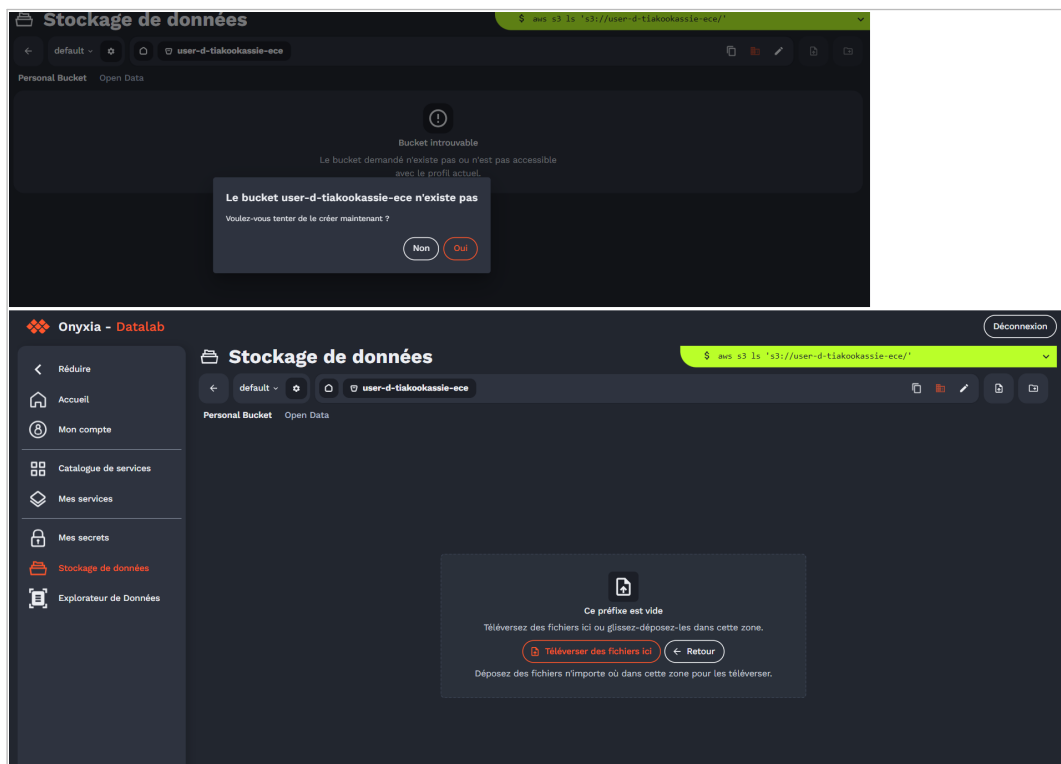


Figure 14 – Création du bucket personnel depuis « Stockage de données »

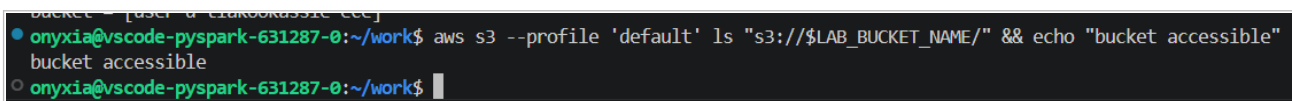


Figure 15 – Bucket accessible depuis la CLI

Étape 2 – Installation de s5cmd

s5cmd est une alternative plus rapide à aws s3 pour les transferts en masse (opérations parallèles). Il a été téléchargé depuis GitHub, son empreinte SHA-256 vérifiée, puis installé dans ~/.local/bin.

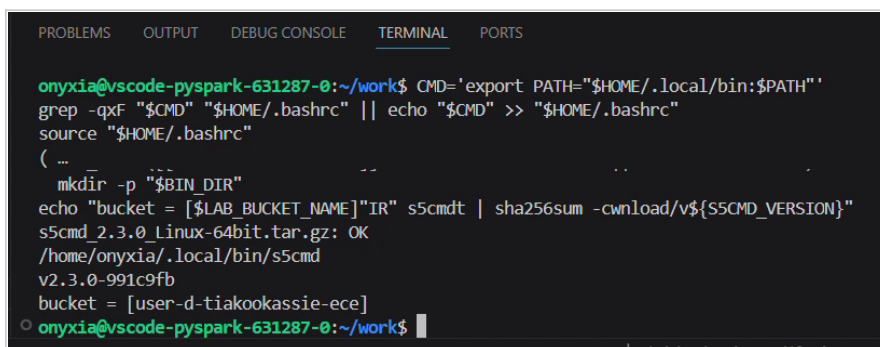


Figure 16 – s5cmd v2.3.0 installé, empreinte vérifiée (OK)

Étape 3 – Upload, liste et téléchargement

Un upload est une requête HTTP **PUT** atomique. Le préfixe bronze/ n'est pas un dossier : le « / » fait simplement partie de la clé dans un espace de noms plat.

```
onyxia@vscode-pyspark-631287-0:~/work$ cd /home/onyxia/work
uv run dataset-users -o csv > users.csv
aws s3 --profile 'default' cp users.csv "s3://$LAB_BUCKET_NAME/bronze/users.csv"
aws s3 --profile 'default' ls "s3://$LAB_BUCKET_NAME/bronze/" ...
diff users.csv users_downloaded.csv && echo "identical"
upload: ./users.csv to s3://user-d-tiakookassie-ece/bronze/users.csv
2026-10-02 12:51:38      7351 users.csv
2026/10/02 12:51:38      7351 users.csv
download: s3://user-d-tiakookassie-ece/bronze/users.csv to ./users_downloaded.csv
identical
onyxia@vscode-pyspark-631287-0:~/work$
```

Figure 17 – PUT, liste (aws et s5cmd) puis GET : fichiers identiques, 7 351 octets

Déplacement et suppression : S3 ne propose pas de renommage atomique. `aws s3 mv` effectue une copie puis une suppression ; un pipeline peut donc observer brièvement les deux clés, ou un état partiel en cas d'échec.

Étape 4 – Métadonnées

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
onyxia@vscode-pyspark-631287-0:~/work$ # Déplacer / renommer = copie + suppression
aws s3 --profile 'default' mv "s3://$LAB_BUCKET_NAME/bronze/users.csv" "s3://$LAB_BUCKET_NAME/bronze/users_renamed.csv"
aws s3 --profile 'default' ls "s3://$LAB_BUCKET_NAME/bronze/"
...
{
  "AcceptRanges": "bytes",
  "LastModified": "2026-10-02T12:53:14+00:00",
  "ContentLength": 7351,
  "ETag": "\"6a28331508e96326f4ae39eac53d02a8\"",
  "ContentEncoding": "aws-chunked",
  "ContentType": "text/csv",
}

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
onyxia@vscode-pyspark-631287-0:~/work$ # Déplacer / renommer = copie + suppression
aws s3 --profile 'default' mv "s3://$LAB_BUCKET_NAME/bronze/users.csv" "s3://$LAB_BUCKET_NAME/bronze/users_renamed.csv"
aws s3 --profile 'default' ls "s3://$LAB_BUCKET_NAME/bronze/"
...
  "ContentEncoding": "aws-chunked",
  "ContentType": "text/csv",
  "Metadata": {}
}
6a28331508e96326f4ae39eac53d02a8 users.csv
upload: ./users.csv to s3://user-d-tiakookassie-ece/bronze/users.csv
{
```

Figure 18 – head-object : ETag égal au MD5 du fichier ; Content-Type text/csv

Résultat : l'ETag `6a28331508e96326f4ae39eac53d02a8` est identique au md5sum local, comme attendu pour un upload en une seule requête. Les métadonnées utilisateur (source, version, rows) ont été attachées lors d'un nouvel upload : un objet et ses métadonnées sont immuables, et les modifier impose de réécrire l'objet.

Écarts observés (SeaweedFS) : l'ETag est affiché entre guillemets doublés, et le champ `ContentEncoding: aws-chunked` apparaît, lié à l'envoi par morceaux avec somme de contrôle des versions récentes de la CLI. Le contenu n'est pas affecté.

Étape 5 – URL présignées et upload multipart

Une URL présignée embarque la signature et la durée de validité (ici 300 s). `curl`, sans aucun identifiant, a pu lire le fichier. Une URL présignée **PUT** a aussi été générée avec `boto3` : elle permettrait à un tiers de déposer un fichier sans connaître les clés.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

onyxia@vscode-pyspark-631287-0:~/work$ # URL présignée GET, valable 5 minutes
URL=$(aws s3 --profile 'default' presign "s3://$LAB_BUCKET_NAME/bronze/users.csv" --expires-in 300)
echo "$URL"
curl -s "$URL" | head -n 3 ...
29d679010e5fce790dc487f8180a66923dea936
uuid,username,name,sex,address,mail,birthdate
bdd640fb-0667-4ad1-9c80-317fa3b1799d,garzaanthony,Charles Garcia,M,"908 Jennifer Squares
Robinsonchine, KY 01752" bobennetence@gmail.com 1025-00-22

```

Figure 19 – Lecture du fichier via URL présignée GET, sans identifiants

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

onyxia@vscode-pyspark-631287-0:~/work$ # URL présignée GET, valable 5 minutes
URL=$(aws s3 --profile 'default' presign "s3://$LAB_BUCKET_NAME/bronze/users.csv" --expires-in 300)
echo "$URL"
curl -s "$URL" | head -n 3 ...
upload: ./large_dataset.bin to s3://user-d-tiakookassie-ece/large/dataset.bin
"a9bf50569c6c209f2159392536a098d7-25"
{
  "RequestCharged": null,
  "Prefix": ""
}
onyxia@vscode-pyspark-631287-0:~/work$

```

Figure 20 – Upload multipart de 200 Mio : ETag suffixé « -25 » ; aucun upload incomplet

Résultat : au-delà de 8 Mo, la CLI découpe le fichier en parties envoyées en parallèle. 200 Mio / 8 Mio = **25 parties**, d'où le suffixe -25. L'ETag n'est alors plus le MD5 du fichier mais un hash des hashes des parties. Intérêts : reprise partie par partie, parallélisme, et objets jusqu'à 5 To (contre 5 Go pour un PUT simple).

Étape 6 – Versioning

Après activation du versioning (Status: Enabled), deux uploads successifs sur la même clé ont créé deux versions distinctes.

```

onyxia@vscode-pyspark-631287-0:~/work$ aws s3api --profile 'default' list-object-versions --bucket "$LAB_BUCKET_NAME" --prefix bronze/users.csv \
| jq -c '.Versions[] | {VersionId, IsLatest, Size, LastModified}'
{"VersionId":"6725482a690c1c74275356130ee7cf4d","IsLatest":true,"Size":5855,"LastModified":"2026-10-02T12:58:49+00:00"}
{"VersionId":"6725482ae2f4aaaa733186a47d9ae7f9","IsLatest":false,"Size":7351,"LastModified":"2026-10-02T12:58:47+00:00"}
{"VersionId":null,"IsLatest":false,"Size":7351,"LastModified":"2026-10-02T12:53:18+00:00"}
onyxia@vscode-pyspark-631287-0:~/work$

```

Figure 21 – Trois versions de bronze/users.csv

VersionId	IsLatest	Taille	Interprétation
6725482a690c...	true	5 855	Version 2 – 40 utilisateurs
6725482ae2f4...	false	7 351	Version 1 – 50 utilisateurs
null	false	7 351	Objet déposé avant l'activation du versioning

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

onyxia@vscode-pyspark-631287-0:~/work$ aws s3api --profile 'default' get-object \
--bucket "$LAB_BUCKET_NAME" \
--key bronze/users.csv \
--version-id 6725482ae2f4aaaa733186a47d9ae7f9 \
{
  "VersionId": "6725482ae2f4aaaa733186a47d9ae7f9",
  "ContentEncoding": "aws-chunked",
  "ContentType": "text/csv",
  "Metadata": {}
}
version 1 restored
onyxia@vscode-pyspark-631287-0:~/work$

```

Figure 22 – Récupération de la version 1 par son VersionId : « version 1 restored »

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
onyxia@vscode-pyspark-631287-0:~/work$ export AWS_PAGER=""
grep -qF 'export AWS_PAGER=""' ~/.bashrc || echo 'export AWS_PAGER=""' >> ~/.bashrc

aws s3 --profile 'default' rm "s3://$LAB_BUCKET_NAME/bronze/users.csv"
aws s3 --profile 'default' ls "s3://$LAB_BUCKET_NAME/bronze/"
aws s3api --profile 'default' list-object-versions --bucket "$LAB_BUCKET_NAME" --prefix bronze/users.csv \
  | jq -c 'Versions: [.Versions[]? | {VersionId, IsLatest, Size}], DeleteMarkers: [.DeleteMarkers[]? | {VersionId, IsLatest}]'
delete: s3://user-d-tiakookassie-ece/bronze/users.csv
{"Versions":[{"VersionId":"6725482a690c1c74275356130ee7cf4d","IsLatest":false,"Size":5855}, {"VersionId":"6725482ae2f4aaa733186a47d9ae7f9","IsLatest":false,"Size":7351}, {"VersionId":"null","IsLatest":false,"Size":7351}], "DeleteMarkers":[{"VersionId":"672547c82530b65ec23f2afcb6367fa2","IsLatest":true}]}
onyxia@vscode-pyspark-631287-0:~/work$

```

Figure 23 – Après « rm » : l'objet disparaît de la liste, mais les 3 versions restent, sous un delete marker

Analyse : sur un bucket versionné, la suppression n'efface aucune donnée. Elle ajoute un *delete marker* qui devient la version courante : un GET normal échoue, mais toutes les versions restent récupérables. Remarque pratique : la CLI ouvrait ses réponses dans un *pager* bloquant ; il a été désactivé avec `export AWS_PAGER=""`.

Étape 7 – Cycle de vie et contrôle d'accès

Trois règles de cycle de vie ont été appliquées : expiration de large/ après 30 jours, suppression des versions non courantes 7 jours après leur remplacement, et abandon des uploads multipart incomplets après 7 jours. Sur un bucket versionné, une règle *Expiration* ne fait qu'ajouter un delete marker : la règle *NoncurrentVersionExpiration* est indispensable pour maîtriser le coût du stockage des anciennes versions.

Une **bucket policy** interdisant `s3:DeleteObject` sur `bronze/*` a ensuite été appliquée.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
onyxia@vscode-pyspark-631287-0:~/work$ # --- Lifecycle ---
cat > lifecycle.json <<'EOF'
{
  "Rules": [ ...
delete failed: s3://user-d-tiakookassie-ece/bronze/protected.csv An error occurred (AccessDenied) when calling the DeleteObject operation: Access Denied.
upload: ./users.csv to s3://user-d-tiakookassie-ece/bronze/protected.csv
{
  "Owner": {
    "DisplayName": "admin",
    "ID": "admin"
  },
}

```

Figure 24 – Suppression refusée par la bucket policy : AccessDenied

```

onyxia@vscode-pyspark-631287-0:~/work$ aws s3api --profile 'default' get-bucket-lifecycle-configuration --bucket "$LAB_BUCKET_NAME" | jq -c '.Rules[] | {ID, Status}'
aws s3api --profile 'default' get-bucket-policy --bucket "$LAB_BUCKET_NAME"
{"ID":"expire-large-after-30-days","Status":"Enabled"},
{"ID":"delete-noncurrent-versions","Status":"Enabled"},
{"ID":"abort-incomplete-multipart","Status":"Enabled"}

aws: [ERROR]: An error occurred (NoSuchBucketPolicy) when calling the GetBucketPolicy operation: The bucket policy does not exist
onyxia@vscode-pyspark-631287-0:~/work$

```

Figure 25 – Règles de cycle de vie actives ; policy bien retirée (NoSuchBucketPolicy)

Réponse à la question de l'énoncé : oui, la suppression est refusée. Sur cette plateforme, SeaweedFS applique la bucket policy aussi aux utilisateurs authentifiés. La policy a ensuite été supprimée, ce que confirme l'erreur attendue `NoSuchBucketPolicy`. La lecture de l'ACL de l'objet montre un propriétaire interne admin avec `FULL_CONTROL`. Les ACL sont un mécanisme hérité ; les URL présignées restent la méthode recommandée pour partager un objet.

Étape 8 – Cohérence et nettoyage

Le premier test de cohérence a échoué à cause de la destination `/dev/null` : la CLI tente d'en modifier la date, ce qui est interdit. Le test a été refait en écrivant sur la sortie standard (-).

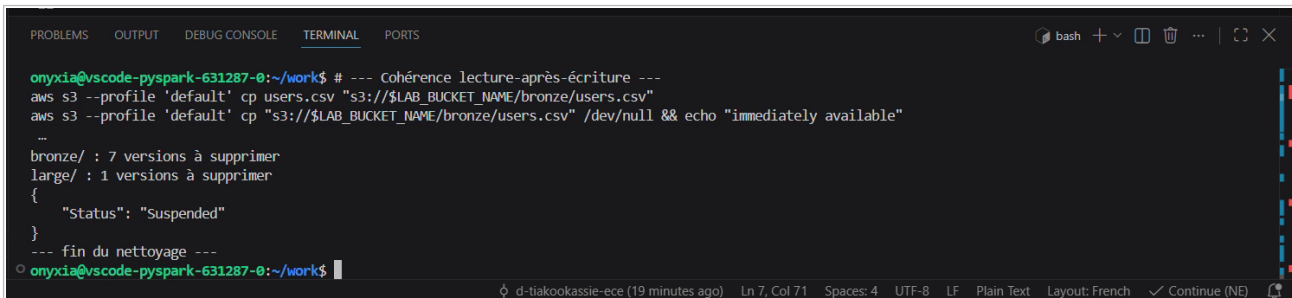
```

onyxia@vscode-pyspark-631287-0:~/work$ aws s3 --profile 'default' cp users.csv "s3://$LAB_BUCKET_NAME/bronze/users.csv"
aws s3 --profile 'default' cp "s3://$LAB_BUCKET_NAME/bronze/users.csv" - > /dev/null && echo "immediately available"
upload: ./users.csv to s3://user-d-tiakookassie-ece/bronze/users.csv
immediately available
onyxia@vscode-pyspark-631287-0:~/work$

```

Figure 26 – Cohérence forte en lecture après écriture : « immediately available »

Le nettoyage supprime explicitement chaque version et chaque delete marker (list-object-versions puis delete-objects), suspend le versioning (il ne peut pas être désactivé une fois activé) et supprime la configuration de cycle de vie.

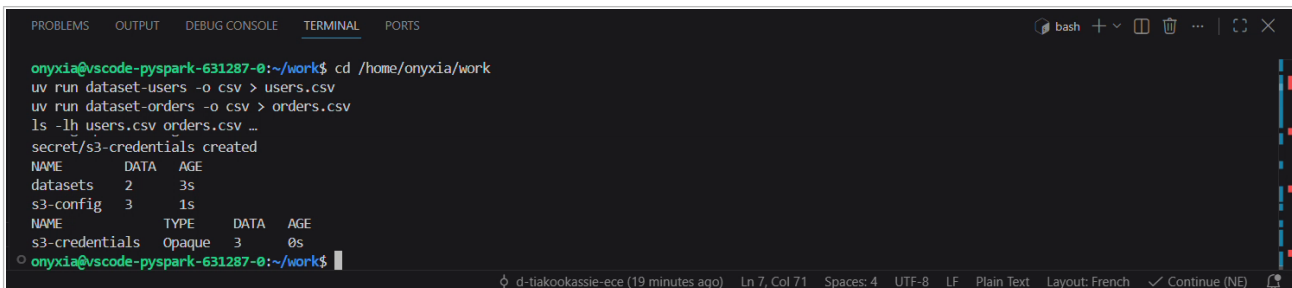


```
onxyxia@vscode-pyspark-631287-0:~/work$ # --- Cohérence lecture-après-écriture ---
aws s3 --profile 'default' cp users.csv "s3://$LAB_BUCKET_NAME/bronze/users.csv"
aws s3 --profile 'default' cp "s3://$LAB_BUCKET_NAME/bronze/users.csv" /dev/null && echo "immediately available"
...
bronze/ : 7 versions à supprimer
large/ : 1 versions à supprimer
{
  "Status": "Suspended"
}
--- fin du nettoyage ---
onxyxia@vscode-pyspark-631287-0:~/work$
```

Figure 27 – Nettoyage : 7 versions supprimées sous bronze/, 1 sous large/ ; versioning suspendu ; bucket vide

Étape 9 – Ingestion dans la couche bronze par un Job Kubernetes

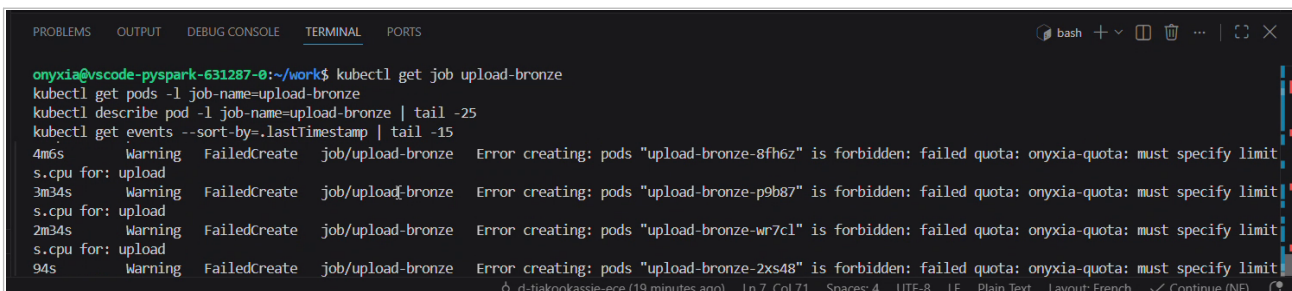
Les deux jeux de données sont placés dans une ConfigMap datasets (montée en fichiers dans /data), la configuration non sensible dans la ConfigMap s3-config, et les identifiants dans le Secret s3-credentials, exportés via un fichier temporaire supprimé aussitôt et jamais affichés. Le Job exécute l'image officielle amazon/aws-cli.



```
onxyxia@vscode-pyspark-631287-0:~/work$ cd /home/onxyxia/work
uv run dataset-users -o csv > users.csv
uv run dataset-orders -o csv > orders.csv
ls -lh users.csv orders.csv ...
secret/s3-credentials created
NAME      DATA   AGE
datasets  2       3s
s3-config  3       1s
NAME      TYPE    DATA   AGE
s3-credentials  Opaque  3       0s
onxyxia@vscode-pyspark-631287-0:~/work$
```

Figure 28 – ConfigMaps (2 et 3 entrées) et Secret Opaque (3 entrées) créés

Difficulté : le Job est resté bloqué (timed out waiting for the condition), sans qu'aucun Pod ne soit créé. Les événements du namespace ont révélé la cause :



```
onxyxia@vscode-pyspark-631287-0:~/work$ kubectl get job upload-bronze
kubectl get pods -l job-name=upload-bronze
kubectl describe pod -l job-name=upload-bronze | tail -25
kubectl get events --sort-by=.lastTimestamp | tail -15
4m6s      Warning   FailedCreate   job/upload-bronze   Error creating: pods "upload-bronze-8fh6z" is forbidden: failed quota: onxyxia-quota: must specify limits
s.cpu for: upload
3m34s     Warning   FailedCreate   job/upload-bronze   Error creating: pods "upload-bronze-p9b87" is forbidden: failed quota: onxyxia-quota: must specify limits
s.cpu for: upload
2m34s     Warning   FailedCreate   job/upload-bronze   Error creating: pods "upload-bronze-wr7cl" is forbidden: failed quota: onxyxia-quota: must specify limits
s.cpu for: upload
94s       Warning   FailedCreate   job/upload-bronze   Error creating: pods "upload-bronze-2xs48" is forbidden: failed quota: onxyxia-quota: must specify limits
onxyxia@vscode-pyspark-631287-0:~/work$
```

Figure 29 – Pods refusés par le quota onxyxia-quota : « must specify limits.cpu »

Le namespace est soumis à un **ResourceQuota** qui impose une limite CPU à chaque conteneur ; or le manifeste de l'énoncé ne fixe qu'une limite mémoire. Le modèle de Pod d'un Job étant immuable, le Job a été supprimé, puis recréé avec une limite CPU. Le style d'adressage *path* a également été forcé dans le conteneur, qui ne dispose pas du fichier ~/ .aws/config.

```
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m          # ajout imposé par le quota onxyxia-quota
    memory: 256Mi
...
args:
- |
  set -e
  aws configure set default.s3.addressing_style path # ajout
```

```

for dataset in users orders; do
aws s3 cp "/data/$dataset.csv" "s3://$LAB_BUCKET_NAME/bronze/$dataset.csv"
done

```

```

onxyia@vscode-pyspark-631287-0:~/work$ kubectl delete job upload-bronze
sed -i 's/^(.*)limits:$/&\n1  cpu: 500m/' job-upload-bronze.yaml
grep -A3 'limits:' job-upload-bronze.yaml
...
upload: ../data/orders.csv to s3://user-d-tiakookassie-ece/bronze/orders.csv
2026-10-02 13:25:33    331568  bronze/orders.csv
2026-10-02 13:25:28    7351    bronze/users.csv
--- vérification depuis le terminal ---
2026-10-02 13:25:33    331568  bronze/orders.csv
2026-10-02 13:25:28    7351    bronze/users.csv
onxyia@vscode-pyspark-631287-0:~/work$

```

Figure 30 – Job terminé : bronze/orders.csv (331 568 o) et bronze/users.csv (7 351 o)

Résultat : les tailles sont identiques à celles de l'énoncé. Les objets Kubernetes ont ensuite été supprimés ; les fichiers de la couche bronze sont conservés pour les modules suivants. Le manifeste corrigé a été versionné dans le dépôt.

```

onxyia@vscode-pyspark-631287-0:~/work$ kubectl delete -f job-upload-bronze.yaml
kubectl delete configmap datasets s3-config
kubectl delete secret s3-credentials
kubectl get jobs,configmaps,secrets | grep -E 'upload-bronze|datasets|s3-' || echo "objets du lab supprimés" ...
configmap "datasets" deleted from user-d-tiakookassie-ece namespace
configmap "s3-config" deleted from user-d-tiakookassie-ece namespace
secret "s3-credentials" deleted from user-d-tiakookassie-ece namespace
objets du lab supprimés
2026-10-02 13:25:33    331568  orders.csv
2026-10-02 13:25:28    7351    users.csv
onxyia@vscode-pyspark-631287-0:~/work$

onxyia@vscode-pyspark-631287-0:~/work$ git add job-upload-bronze.yaml
git commit -m "feat: kubernetes job uploading datasets to bronze"
git push
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 785 bytes | 785.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Dorline0121/ece-2026-bigdata.git
e8c41e3..134d464  main -> main
onxyia@vscode-pyspark-631287-0:~/work$

```

Figure 31 – Suppression des objets Kubernetes, données bronze conservées, manifeste poussé sur GitHub

Particularités de la plateforme Onyxia contournées

Quatre écarts entre l'énoncé et l'environnement réel ont nécessité une adaptation. Chacun illustre un mécanisme du cours.

#	Symptôme	Cause	Solution appliquée
1	« The config profile (default) could not be found »	Onyxia fournit les identifiants S3 uniquement en variables d'environnement, sans fichiers ~/.aws	Création du profil default avec aws configure set à partir des variables AWS_* ; endpoint sans « / » final ; addressing_style path
2	« Bucket introuvable » ; aucun bucket personnel listé	Le bucket user-<namespace> n'est créé qu'à la première ouverture de l'explorateur de fichiers	Ouverture de « Stockage de données » puis création du bucket depuis le portail
3	Job bloqué ; FailedCreate « must specify limits.cpu »	ResourceQuota onyxia-quota sur limits.cpu dans le namespace	Ajout de limits.cpu: 500m, suppression puis recréation du Job
4	Risque d'URL virtual-hosted (bucket.endpoint) dans le conteneur	L'image aws-cli n'a pas le profil configuré et utilise le style d'adressage par défaut	aws configure set default.s3.addressing_style path dans le script du Job

Autres ajustements mineurs

- `git config --global --add safe.directory /home/onyxia/work` pour l'erreur « dubious ownership » liée au volume monté.
- `export AWS_PAGER=""` pour éviter le blocage du terminal sur « (END) ».
- Test de cohérence vers la sortie standard plutôt que `/dev/null`, que la CLI ne peut pas horodater.
- Identifiants temporaires : en cas d'ExpiredToken, redémarrer le service, puis recréer le profil et les variables.

Réponses aux questions du lab 2

1. Pourquoi les identifiants sont-ils stockés dans un Secret et non dans la ConfigMap ?

Une ConfigMap est destinée à la configuration non sensible : son contenu est lisible en clair avec `kubectl get` ou `describe` par toute personne autorisée à lire les ConfigMaps du namespace. Un Secret est un type d'objet distinct, soumis à ses propres règles RBAC : on peut autoriser la lecture des ConfigMaps sans donner accès aux Secrets. Il n'est pas affiché par défaut, peut être chiffré au repos dans etcd, et il est monté en mémoire (tmpfs) plutôt que sur disque. L'endpoint, la région et le nom du bucket vont donc dans `s3-config` ; les clés et le jeton de session dans `s3-credentials`.

2. Que se passe-t-il si le Job est relancé demain ? Comment une plateforme de production fournirait-elle les identifiants ?

Les identifiants Onyxia sont temporaires (jeton de session). Demain, le Secret contiendra des clés expirées et le Job échouera avec `ExpiredToken` ou `InvalidAccessKeyId`, sauf à recréer le Secret à la main. En production, on ne copie pas de clés manuellement dans un Secret :

- **Identité de charge de travail** (IRSA ou EKS Pod Identity, Workload Identity sur GCP ou Azure) : le ServiceAccount du Pod est associé à un rôle, et le Pod reçoit automatiquement des jetons de courte durée, renouvelés en continu, sans aucun secret stocké.
- **Gestionnaire de secrets** (HashiCorp Vault, External Secrets Operator) : génération dynamique ou rotation régulière des identifiants, avec synchronisation du Secret Kubernetes.

-
- **Provisionnement par opérateur** : avec Rook/Ceph, un `ObjectBucketClaim` crée le bucket ainsi qu'un `Secret` et une `ConfigMap` dédiés, injectés dans le Pod.

3. Comment transformer ce Job en ingestion quotidienne ?

Avec un **CronJob**, qui crée un Job selon une planification de type cron. Le modèle de Pod (y compris `limits.cpu`) est repris sous `jobTemplate` :

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: upload-bronze-daily
spec:
  schedule: "0 2 * * *"          # chaque jour à 2 h
  concurrencyPolicy: Forbid      # pas d'exécutions concurrentes
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        # même spécification de Pod que le Job
```

Pour une véritable ingestion, il faudrait aussi :

- **partitionner par date** (par exemple `bronze/orders/date=2026-10-02/orders.csv`), pour ne pas écraser les données de la veille et faciliter le *partition pruning* ;
- **lire depuis le système source** plutôt que depuis une `ConfigMap`, limitée à 1 Mio ;
- **utiliser des identifiants renouvelés automatiquement** (question 2) ;
- pour des pipelines avec dépendances, reprise et supervision, passer par un orchestrateur (Airflow, Argo Workflows).

Conclusion

Les deux labs ont été menés à terme. Le projet uv génère des jeux de données reproductibles et liés, versionnés sur GitHub. Le lab S3 a permis de vérifier concrètement les propriétés du stockage objet : objets immuables adressés par clé dans un espace plat, absence de renommage atomique, ETag et multipart, versioning et delete markers, cycle de vie, politiques d'accès et cohérence forte. Les écarts rencontrés (profil absent, bucket à créer, quota CPU, style d'adressage) ont été diagnostiqués à partir des messages d'erreur et des événements Kubernetes, puis corrigés. Les jeux de données `users.csv` et `orders.csv` sont désormais disponibles dans la couche bronze pour les modules suivants.